

Programming Basics 1



07: Pakete und Innere Klassen

Sommersemester 2024

Simon Dewert

HAW Hamburg

Fakultät Design, Medien und Information

E52, Finkenau 35, 22081 Hamburg

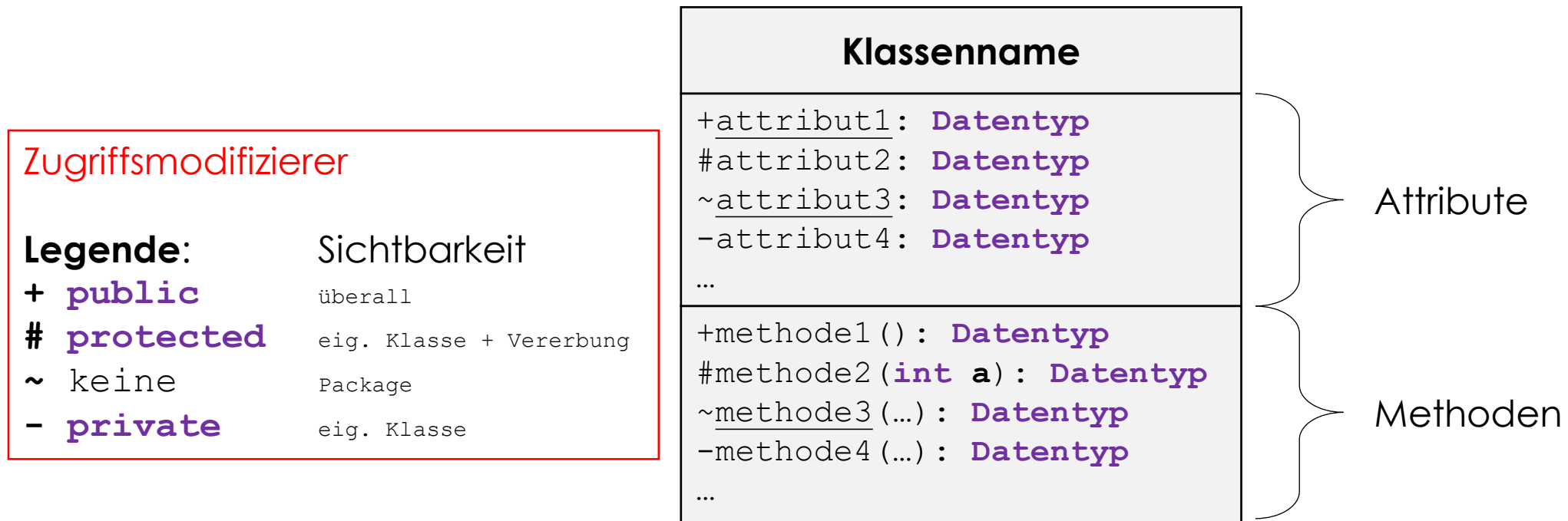
simon.dewert@haw-hamburg.de

Übersicht dieser Lehreinheit

1. Nachtrag
 - a. UML 2.0 Klassendiagramm
 - b. UML 2.0 Aktivitätsdiagramm für Methoden
 - c. Gültigkeitsbereiche von Variablen
2. Pakete
3. Zuordnung zu Paketen
4. Zugriff auf Klassen in Paketen
5. Zugriff auf statische Elemente in Paketen
6. UML 2.0 Klassendiagramm: Pakete
7. Innere Klassen
8. Innere Klassen: Zugriffe
9. Lokale Klassen

Nachtrag: UML 2.0 Klassendiagramm

Mit einem UML 2.0 Klassendiagramm können Klassen grafisch dargestellt werden:



Hinweise: Sind Elemente unterstrichen , so sind diese **static**!

Nachtrag: Gültigkeitsbereiche von Variablen

Ein eine Variable bzw. ein Attribut kann nur innerhalb ihres bzw. seines Gültigkeitsbereichs verwendet werden:

```
class EineKlasse
{
    int zahlEins = 0;
    void eineMethode(int zahlZwei)
    {
        int zahlEins = 1;
        for (int zahlDrei = 0; zahlDrei < 10; zahlDrei++)
        {
            if (zahlDrei < 5)
            {
                int zahlVier = this.zahlEins + zahlEins + zahlZwei + zahlDrei;
            }
        }
    }
}
```

Verdeckung

Scope Analysis:

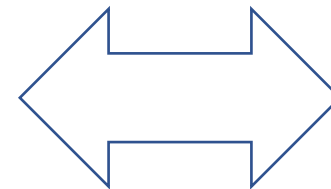
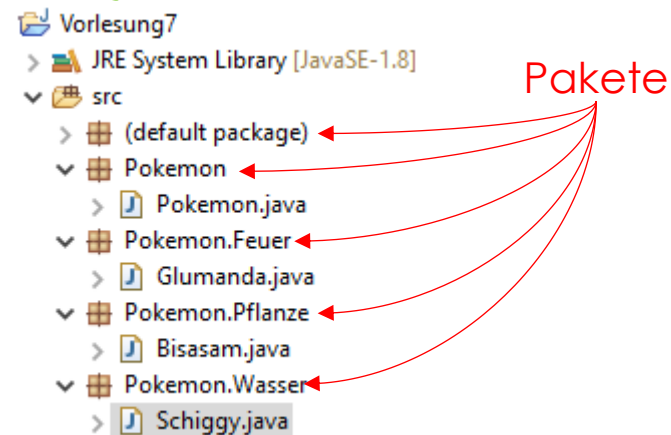
- Class Scope:** `this.zahlEins`
- Method Scope:** `zahlEins, zahlZwei` and `this.zahlEins`
- For Loop Scope:** `zahlDrei` and `this.zahlEins, zahlEins, zahlZwei`
- If Statement Scope:** `zahlVier` and `this.zahlEins, zahlEins, zahlZwei, zahlDrei`

Pakete

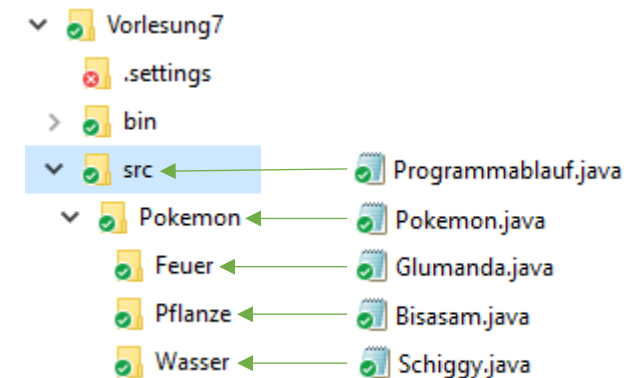
Pakete organisieren Klassen in einer **hierarchischen Struktur**, um **Ordnung** zu schaffen, **Namenskonflikte** zu vermeiden und bestimmte Klassen und ihre Elemente durch Verwendung geeigneter Zugriffsmodifizierer zu **verbergen**. Technisch entspricht die Organisation von Klassen in Pakete und Unterpakete der Organisation der entsprechenden Dateien in **Ordner** und **Unterordner**.

Beispiel:

Projektstruktur



Verzeichnisstruktur



Eine Klasse einem Paket zuordnen

Um eine Klasse einem Paket zuzuordnen, ist das Paket **vor** Deklaration der Klasse unter Einsatz des Schlüsselwortes **package** zu nennen.

Beispiele:

```
package Pokemon.Feuer;
```

```
class Glumanda  
{  
    [...]  
}
```

```
package Pokemon.Wasser;
```

```
class Schiggy  
{  
    [...]  
}
```

Ohne explizite Angabe eines Pakets gehört eine Klasse automatisch dem sogenannten **Default-Package** an.

Zugriff auf eine Klasse eines Pakets

Um auf eine **Klasse** eines bestimmten Pakets zuzugreifen gibt es zwei Möglichkeiten:

1. Importieren der Klasse über das Schlüsselwort **import**:

```
import Pokemon.Feuer.Glumanda;  
[...]  
Glumanda glum = new Glumanda();
```

2. Angabe des "vollqualifizierten" Namens der Klasse und des Konstruktors:

```
Pokemon.Feuer.Glumanda glum2 = new Pokemon.Feuer.Glumanda();
```

Wichtig: Sowohl die Klasse selbst als auch der aufgerufene Konstruktor müssen öffentlich (**public**) sein, es sei denn der Zugriff geschieht innerhalb desselben Pakets!

Zugriff auf alle Klasse eines Pakets

Um auf alle Klassen eines Pakets zuzugreifen ist beim Importieren des Pakets ein Sternchen (*) zu verwenden: Importieren der Klasse über das Schlüsselwort **import**:

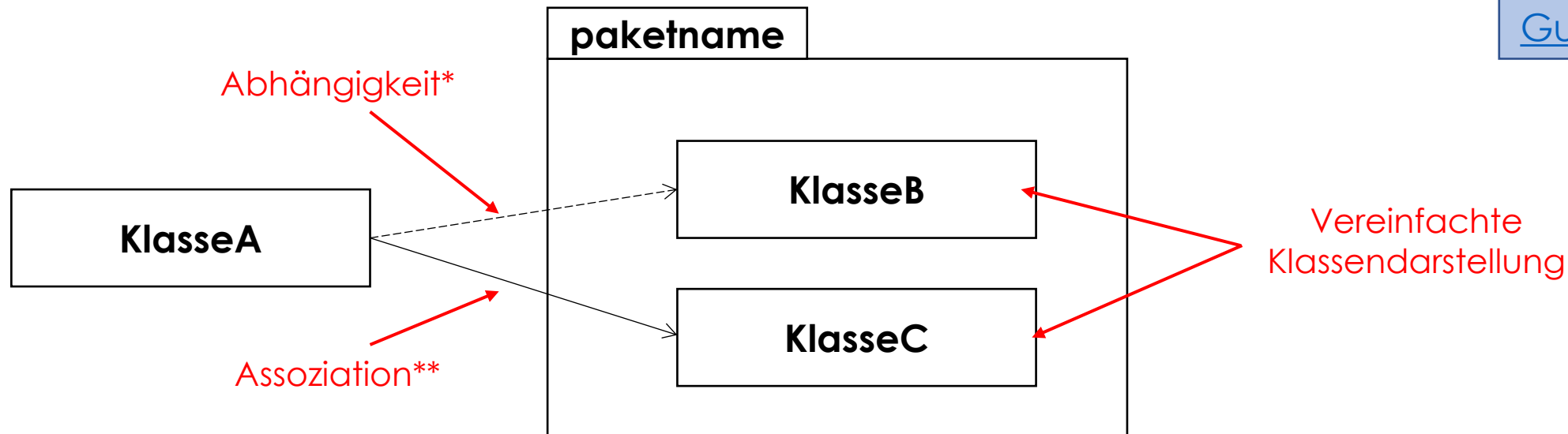
```
import Pokemon.Feuer.*;
[...]  
Glumanda glum = new Glumanda();  
  
// Instanziierung weiterer Klassen des Pakets Pokemon.Feuer  
  
[...]
```

Hinweis: Es werden nur alle Klassen innerhalb des angegebenen Pakets importiert. Klassen in **untergeordneten** Paketen werden hierdurch **nicht** mitimportiert!

UML 2.0 Klassendiagramm: Pakete

Auch Pakete lassen sich in UML 2.0 Klassendiagrammen abbilden. Hinzu kommen Pfeile zur Darstellung der Beziehung Klassen zueinander:

[Gut erklärtes Video](#)

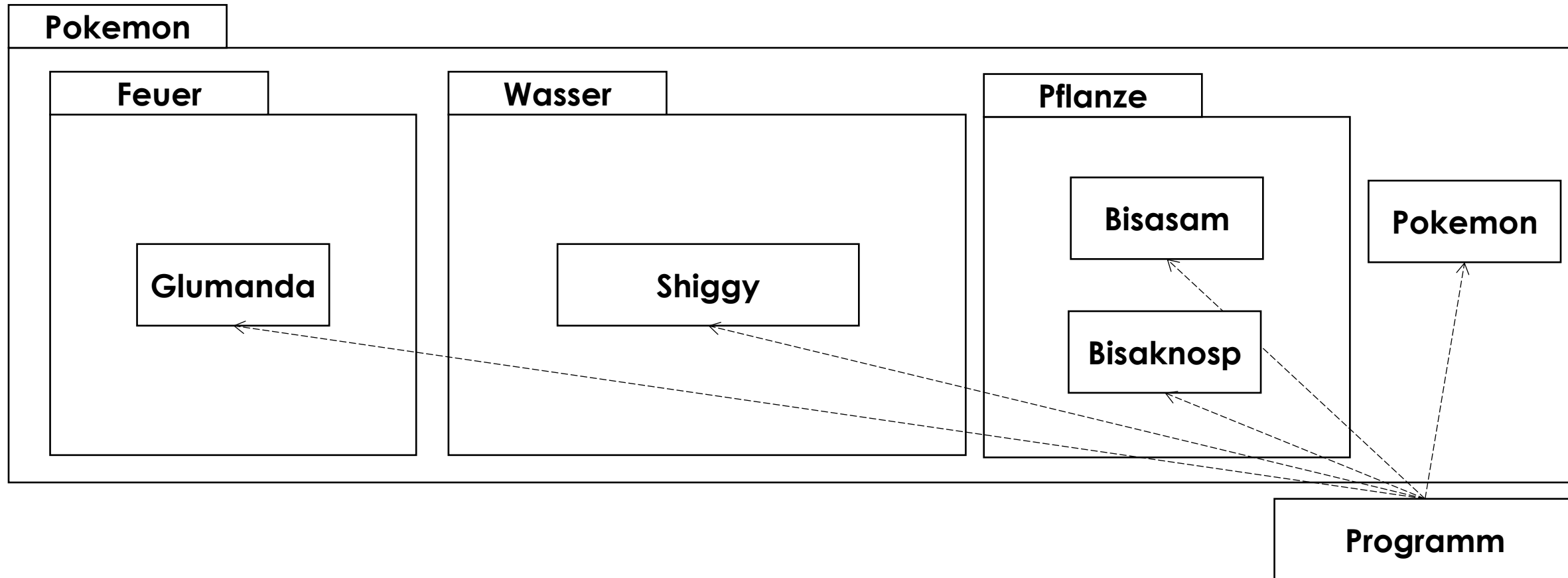


*Abhängigkeit: KlasseA verwendet KlasseB

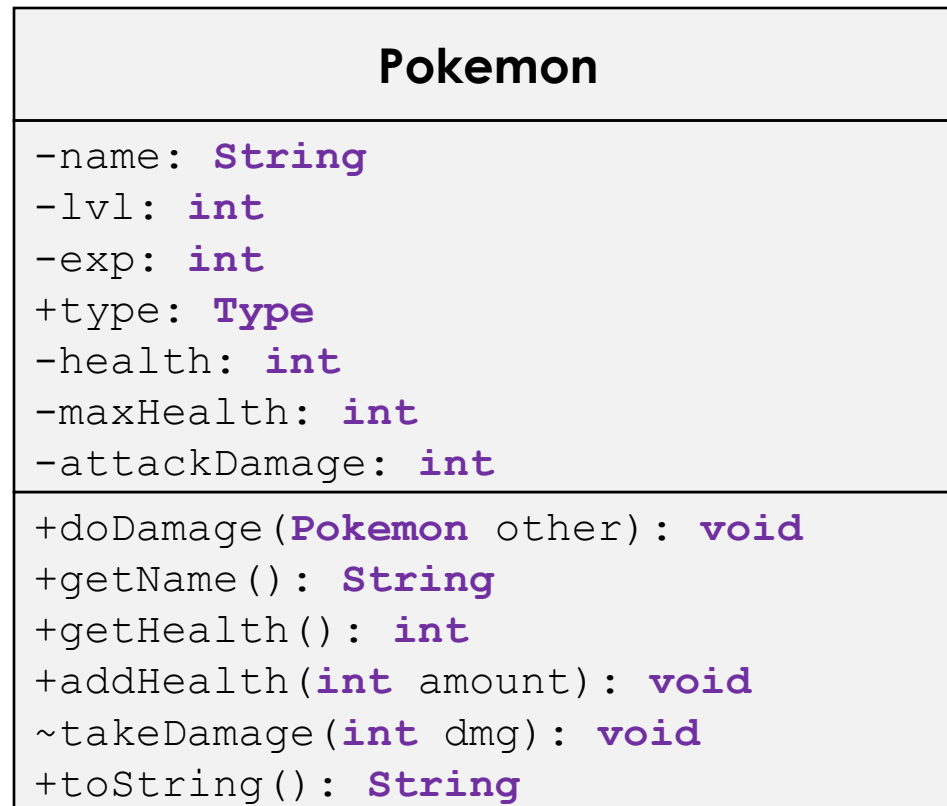
** Assoziation: KlasseA hat ein oder mehrere **Attribute** von Typ KlasseC

UML 2.0 Klassendiagramm: Pakete

UML 2.0 Klassendiagramm des aktuellen Programms:



UML: Klassendiagramm (Pokemon)



Aufgabe 7.1



Wir bauen gemeinsam ein neues Projekt **vorlesung_07**, welches das Klassendiagramm von Seite 12 abbildet.

- Die Klassen Glumanda, Schiggy, Bisasam und Bisaknosp können **leer** bleiben, sollen aber in die richtigen Pakete (siehe S.11)
- Erzeugen Sie alle Attribute anhand des Klassendiagramms.
- Fügen Sie für Pokemon einen Konstruktor.
- Implementieren Sie die Methoden ihrem Namen entsprechend.

Bsp:

`takeDamage(int dmg)` reduziert die health um den übergebenen Wert `dmg`

Pokemon	
B	-name: String
	-lvl: int
	-exp: int
	+type: Type
C	-health: int
	-maxHealth: int
	-attackDamage: int
	+Pokemon(String n, Type t, int hp)
D	+getName(): String
	+getHealth(): int
	+addHealth(int amount): void
	~takeDamage(int dmg): void
	+doDamage(Pokemon other): void
	+toString(): String

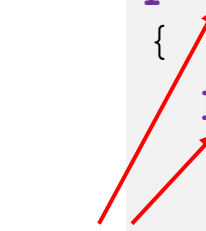
Type kann als Enum umgesetzt werden

Innere Klasse

Innere Klassen sind Klassen, die als **Element einer Klasse** definiert sind:

```
public class MeineKlasse           // Äußere Klasse (Top-Level Klasse)
{
    public class InnereKlasse      // Innere Klasse
    {
        [...]
    }
    [...]
}
```

optional



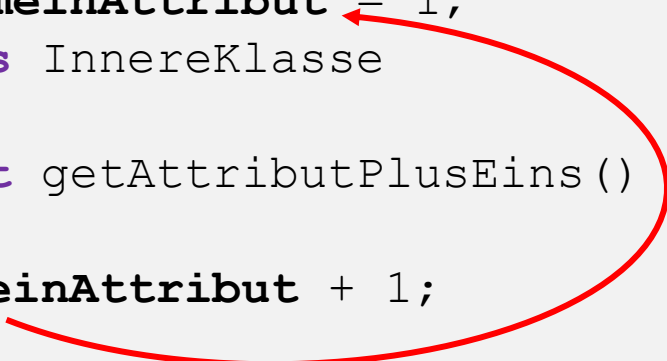
Innere Klassen werden oft als **Hilfsklassen** einer (äußeren) Klasse verwendet.

Hierzu können Sie im Gegensatz zu (normalen) Top-Level Klassen durch **private** und **protected** vor Zugriffen anderer als ihrer äußeren Klasse geschützt werden.

Innere Klasse: Zugriff "nach oben"

Eine innere Klasse kann auf alle (auch die als **private** deklarierten) Elemente ihrer äußeren Klasse **direkt** zugreifen:

```
public class MeineKlasse
{
    private int meinAttribut = 1;
    private class InnereKlasse
    {
        private int getAttributPlusEins()
        {
            return meinAttribut + 1;
        }
    }
}
```



Zugriff auf ein
Attribut der
äußeren Klasse

Innere Klasse: Zugriff "von oben"

Eine äußere Klasse kann nur über ein Objekt der Inneren Klasse auf die Attribute der inneren Klasse zugreifen.

```
public class MeineKlasse
{
    private class InnereKlasse
    {
        private int zahlZehn = 10;
        private int getZwanzig() { return 20; }
    }

    private int getSumme()
    {
        InnereKlasse i = new InnereKlasse();
        return i.zahlZehn + i.getZwanzig();
    }
}
```

Zugriff auf eine
private Methode
der inneren Klasse

Zugriff auf ein
private Attribut der
inneren Klasse

Aufgabe 7.2



a. Schreiben Sie für die Klasse **Pokemon** eine **innere Klasse Attack**, nach dem folgenden Klassendiagramm + **Konstruktor**.

b. **toString()** soll Namen der Attacke, Schadenswert und Typ zurückgeben.

c. Pokemon soll jetzt statt dem Attribut **attackDamage** eine **Liste** (allAttacks) mit möglichen Attacken bekommen (typ **Attack**)

d. Pokemon Konstruktor soll der neuen Liste die Attacke Tackle hinzufügen.

e. Die Methode **doDamage()** soll sich eine **zufällige Attacke aus der Liste** wählen und für den Angriff nutzen.

Pokemon.Attack
-name: String
-damage: int
-type: Type
+toString(): String

Mögliche Ausgabe:
Tackle (10 NORMAL)

NORMAL im Enum
Type ergänzen

Innere Klasse: Zugriff von außen

Eine innere Klasse kann, sofern sie **nicht** als **private** deklariert ist, auch von anderen als der äußeren Klasse verwendet werden.

Hierzu ist jedoch die innere Klasse **über ein Objekt** der äußeren Klasse zu instanziiieren:

```
public class MeineKlasse
{
    public class InnereKlasse
    {
    }
}
```

Zu beachten!

```
MeineKlasse meinObjekt = new MeineKlasse();
MeineKlasse.InnereKlasse meinInneresObjekt = meinObjekt.new InnereKlasse();
```



Lokale Klassen

Auch Methoden können **eigene innere Klassen** haben. Diese werden lokale Klassen genannt und können nur innerhalb der Methode verwendet werden. Sie können auch nicht auf Elemente der übergeordneten Klasse zugreifen.

Aufgrund der oben genannten Eigenschaften werden lokale Klassen stets **ohne Zugriffsmodifizierer** definiert:

```
void meineMethode ()
{
    class LokaleKlasse
    {
    }
    LokaleKlasse lokalesObjekt = new LokaleKlasse ();
}
```

Aufgabe 7.3



Fügen Sie dem Paket **Pokemon** eine Klasse **Trainer** nach dem angegebenen Klassen-Diagramm hinzu:

catchPokemon() soll das übergebene Pokemon der Liste **allMyPokemon** hinzufügen.

callPokemon() soll das erste **kampfbereite** (*health* > 0) Pokemon aus der Liste **allMyPokemon** auswählen und dieses in **activePokemon** ablegen.

Trainer
+name: String
-allMyPokemon: List<Pokemon>
+activePokemon: Pokemon
+catchPokemon(Pokemon poke): void
~callPokemon(): void

Fügen Sie außerdem der Klasse **Pokemon** ein **public Attribut vom Typ Trainer** hinzu. Ein **Pokemon**, welches bereits einen **Trainer** hat, soll nicht von einem anderen gefangen werden können.

Viele Dank für Ihre Aufmerksamkeit



Hausaufgabe



Im bestehenden Pokemon-Projekt, soll die Klasse Trainer folgende Methoden bekommen.

```
+fight(Trainer enemy): void  
+isReadyToFight(): boolean
```

IsReadyToFight() soll **true** zurück liefern, wenn noch mindestens ein Pokemon am Leben ist.

Erstellen Sie in Ihrer main-Methode zwei Trainer und einige Pokemon. Verteilen Sie die Pokemon mit Hilfe von **catchPokemon()** auf die beiden Trainer.

Lassen Sie die Trainer in einer Schleife immer wieder gegeneinander antreten, bis einer nicht mehr „readyToFight“ ist